Este documento presenta una **arquitectura basada en estados y persistencia en Base de Datos (DB)** para procesar y adaptar PDFs de forma asíncrona.

La premisa clave del documento es la siguiente: **La "verdad" de la aplicación vive en el Backend (Base de Datos) y no en la memoria del navegador (sessionStorage)**. Esto significa que cualquier cambio de pantalla, recarga de página o navegación mantiene el estado intacto.

Concepto Clave: El "Análisis" (analysisId) como Hilo Conductor

Todo el proceso gira en torno a un recurso principal: el **Análisis** (o Job). Cuando subís un PDF, el backend crea un ID único (ej. analysisId: "abc-123"). Este ID se pasa por la URL en todas las pantallas.

Desglose Paso a Paso del Flujo (Frontend $ \leftrightarrow$ Backend)

Paso 1: Cargar (/analisis/cargar)

- **Qué hace el Frontend:** El usuario selecciona el PDF en un formulario y se envía mediante POST /analyses usando multipart/form-data.
- **Qué hace el Backend:**

1. Recibe el archivo y lo guarda en un almacenamiento de objetos (como AWS S3 o MinIO).
2. Crea un registro en la base de datos en la tabla Analysis con estado status = "uploaded".
3. Encola una tarea asíncrona (usando herramientas como *BullMQ*, *Celery* o *Cloud Tasks*) para procesar el PDF en segundo plano.
4. Responde inmediatamente al Frontend con el { analysisId } sin hacer esperar al usuario.

- **El Frontend pasa al siguiente paso:** Guarda ese analysisId en la URL (/analisis/analizar?analysisId=abc-123).

Paso 2: Analizar (/analisis/analizar - Estado de Carga)

Dado que procesar un PDF con Inteligencia Artificial puede demorar varios segundos, no se puede resolver en una sola petición HTTP síncrona.

- **Qué hace el Backend en Background (Workers):**

1. Extrae el texto y la estructura del PDF.
2. Ejecuta los algoritmos de detección de barreras por dimensión (comprensión, estructura, elementos visuales, etc.).
3. Guarda en la tabla Barrier cada problema encontrado asociado a ese analysisId.

4. Cambia el estado del análisis a status = "ready".

- **Qué hace el Frontend (Polling o Eventos):**

1. Consulta periódicamente al servidor mediante GET /analyses/:id/status (cada 1 o 2 segundos).
2. Recibe el progreso por dimensión para mostrar una barra de carga o *checklist* animada.
3. *Alternativa:* Puede usar **WebSockets** o **Server-Sent Events (SSE)** mediante GET /analyses/:id/events para recibir notificaciones en tiempo real desde el backend sin necesidad de hacer *polling* repetitivo.
4. Una vez que el estado cambia a "ready", el Frontend navega a la pantalla de **Revisar**.

Paso 3: Revisar (/analisis/revisar?analysisId=...)

- **Qué hace el Frontend:** Realiza una petición GET /analyses/:id/barriers.
- **Qué hace el Backend:** Devuelve la lista completa de barreras detectadas agrupadas por dimensión (comprensión, estructura, etc.) y las estadísticas globales (totales, severidad alta, etc.).
- **Concepto para el Developer:** El frontend no calcula totales ni guarda las barreras en un *state* local persistente; solo consume y renderiza lo que responde el servidor.

Paso 4 y 5: Adaptar y Detalle (/analisis/detalle?analysisId=...&barrierId=...)

En este punto, el usuario interactúa con cada barrera para resolverla.

- **Consultar una barrera específica:** GET /analyses/:id/barriers/:barrierId devuelve los detalles del problema, el contexto donde ocurre y sus recomendaciones.
- **Generar propuesta con IA (LLM):** POST /analyses/:id/barriers/:barrierId/proposals llama a un modelo de IA (como Gemini) para pedir una sugerencia de redacción. Esto consume un intento (remainingAttempts) y guarda la versión en la tabla Proposal.
- **Edición manual (Draft):** Si el usuario modifica el texto a mano, el frontend hace un PUT /analyses/:id/barriers/:barrierId/draft para guardar el borrador en la DB en tiempo real.
- **Aceptar, Descartar o Reabrir:**
- POST .../accept: Marca el estado de la propuesta/barrera como accepted.
- POST .../discard: La marca como discarded.
- POST .../reopen: Permite volver a editar una barrera previamente cerrada.

Paso 6: Resultado y Exportación (/analisis/resultado?analysisId=...)

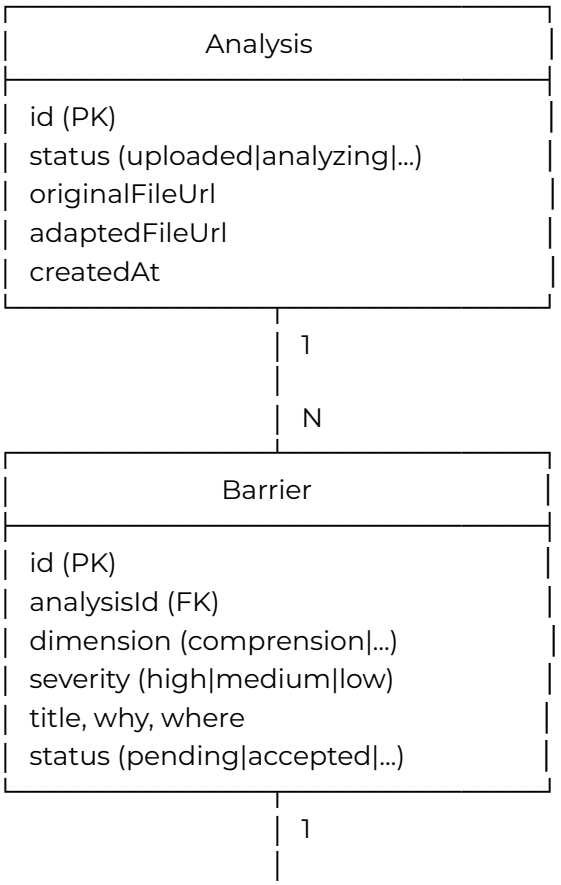
- **Qué hace el Frontend:** Solicita la generación del nuevo archivo adaptado mediante POST /analyses/:id/export.
- **Qué hace el Backend:**

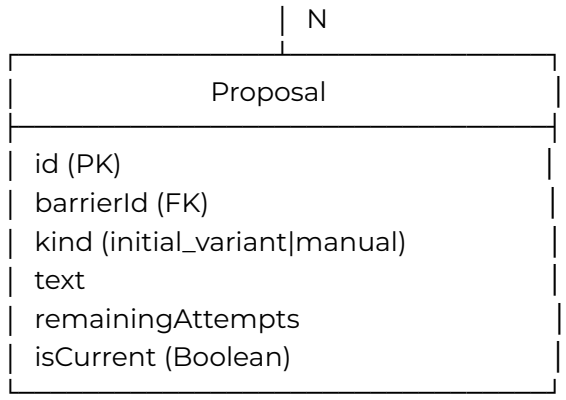
1. Filtra únicamente las barreras cuyo estado sea accepted.
2. Aplica los cambios de texto o estructura sobre el documento original.
3. Genera un nuevo archivo (PDF o DOCX adaptado) y lo sube al almacenamiento (S3/MinIO).
4. Retorna el estado del trabajo de exportación (queued $\rightarrow$ generating $\rightarrow$ ready).

- **Descarga:** El frontend consulta GET /analyses/:id/exports y obtiene la URL para descargar el archivo final listo.

Modelo de Datos Persistido (PostgreSQL / DB Relacional)

La estructura de datos mínima necesaria expuesta en el diseño es la siguiente:





Resumen para el Arquitecto/Developer Full Stack

1. **Desacoplamiento HTTP/Worker:** Subir archivos e invocar llamadas a LLMs son tareas lentas. La API recibe la solicitud, crea registros, encola trabajos en segundo plano y responde de inmediato con IDs.
2. **Navegación declarativa por URL:** Las pantallas se construyen siempre leyendo parámetros de la URL (analysisId, barrierId). Si el usuario refresca la pantalla ($Ctrl+Shift+R$), el frontend vuelve a solicitar los datos al backend mediante un GET y recupera el estado exacto.
3. **API REST Consolidada:** Se necesitan cerca de 12 endpoints para cubrir todas las pantallas sin necesidad de microservicios complejos en etapas tempranas del proyecto.